

Formal verification

Map it to the flow you have done.

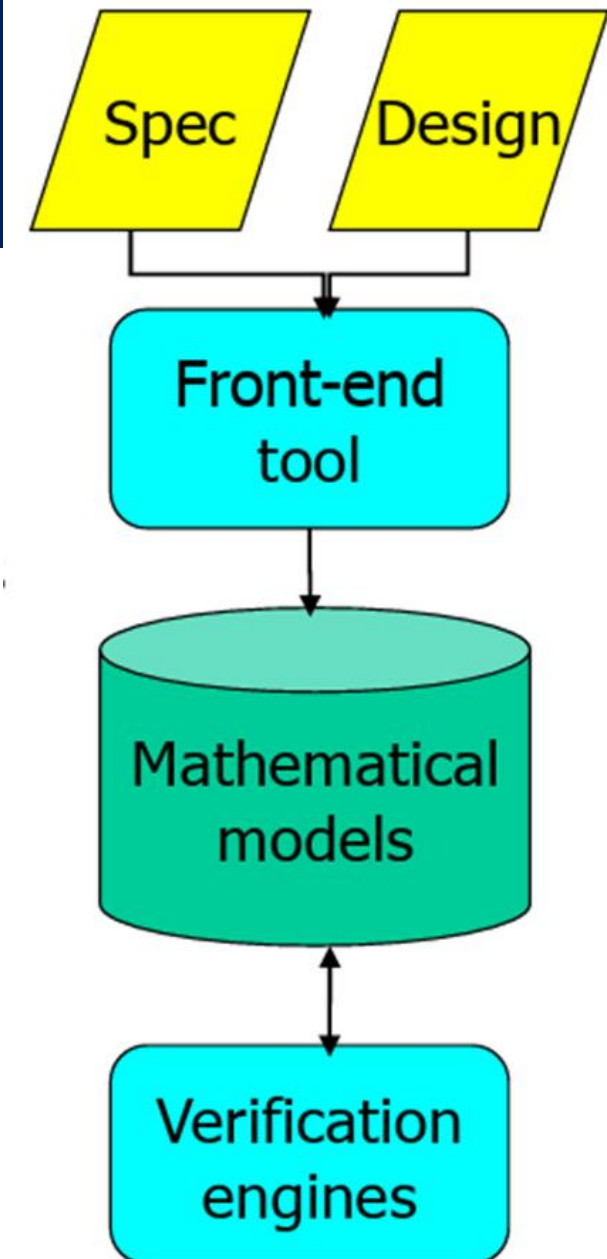
Which is your front end tool.

Which is your back end verification engine. Which is the reasoning engine used in back end.

What is meant by a mathematical model. From theory, give a mathematical model you have studied/give any example.

SAT solver for equivalence and SAT solver for Bounded Model Checking.

What does the output of solver indicate in case of equivalence check and property check?



Question 1. Equivalence checking

You are given two Verilog modules:

Circuit A (reference)

```
module ref(input clk, input a, input b, output reg y);  
always @(posedge clk)  
    y <= a & b;  
endmodule
```

Circuit B (implementation)

```
module impl(input clk, input a, input b, output reg y);  
always @(posedge clk)  
    y <= ~(~a | ~b);  
endmodule
```

Simulation

Using ABC (via Yosys):

Check whether both circuits are **sequentially equivalent**

Give a description of each command you have used for formal verification

Theoretical Work

Model both circuits as transition systems

Construct a miter circuit:

Using SAT-based sequential equivalence checking, determine whether:

$$M_1 \equiv M_2$$

Question 2. Property checking –BMC using SAT solver

Given a counter:

```
module counter(input clk, input rst, output reg [1:0] count);  
always @(posedge clk) begin  
    if (rst)  
        count <= 2'b00;  
    else  
        count <= count + 1;  
    end  
endmodule
```

Simulation

Property to Verify Safety property

The counter should never reach state 11

$G\neg(count = 2'b11)$

Theoretical work

Apply Bounded Model Checking (BMC)

Construct SAT formula:

Give a description of each command you have used for formal verification

In Verilog code have you added anything for property check?

Question 3. Property checking –BMC using SAT solver

For the same 2 bit counter –liveliness check

Property to Verify

Eventually count becomes 00 again

$$GF(count = 00)$$

Give a description of each command you have used for formal verification

Question 4. Optimization and equivalence check

To perform logic optimization on a given RTL design and verify that the optimized netlist is functionally equivalent to the original design using formal equivalence checking.

```
module top(input A, B, C, output F);  
  assign F = (A & B) | (A & B & C);  
endmodule
```

Part A: Logic Optimization

Write a Yosys script to:

Read the RTL file

Prepare the design

Perform logic optimization

Generate the optimized netlist

Give a description of each command you
have used for formal verification

Part B: Equivalence Checking

Write another Yosys script to:

Load the original RTL

Load the optimized netlist

Create a miter circuit

Perform equivalence checking